CHITKARA UNIVERSITY

Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development	**Course Code:** 23EC362
Batch: 2023	**Credits:** 4
Session: Jan-Jun 2026	**Semester:** 6^{th}
Date: 23 Feb 2026	**Class/Group:** IoT Vertical

Lab Evaluation – II

Course Outcome Covered	List of Experiments Evaluated R – Remembering, U – Understanding, P – Applying, N – Analyzing, E – Evaluating, C - Creating	Evaluation Component / marks
CO3, CO5	1. To interface an SSD1306 OLED display over I2C and present real-time sensor values on-screen. 2. To generate PWM signals using a timer and controlling the angular position of a servo motor. 3. To aggregate data from multiple on-board sensors and format it as a structured telemetry frame.	PBL implementation = 15 (U, P, N) Practical File = 5 (U) **Total Marks = 20**

PBL 2: Intelligent Servo Actuation & Telemetry System

Industrial automation systems often combine real-time sensor monitoring with physical actuation, where the movement of a component is conditional upon environmental data. As an embedded engineer, design a system using the STM32L475 IoT node to control a servo motor based on real-time temperature readings and aggregate the telemetry for system diagnostics.

Specifications:

- Configure a timer on the STM32L475 to generate a 50 Hz PWM signal to control the angular position of a servo motor.
- Periodically sample the on-board temperature sensor.
- Implement conditional logic for servo control:
  - SAFE: If temperature < 35°C, move the servo to the 0° position.
  - WARNING: If temperature ≥ 35°C, move the servo to the 90° position.
- Aggregate the temperature data and the current servo angle into a structured telemetry frame (e.g., T:35.2C|ANG:90|STAT:WARNING) and transmit it via USART at 9600 bps at a controlled interval (e.g., every 5 seconds).
- Draw a comprehensive system flowchart showing: timer/PWM initialization, sensor I2C/SPI setup, main loop, condition evaluation, servo position logic, and UART frame formatting.

Evaluation Rubric:

Criterion/Marks	5	3	1

CHITKARA UNIVERSITY

Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development
Course Code: 23EC362
Batch: 2023
Credits: 4
Session: Jan-Jun 2026
Semester: 6th
Date: 23 Feb 2026
Class/Group: IoT Vertical

Lab Evaluation – II

Problem Understanding (Flowchart/system Design)	Complete, well-labeled flowchart showing peripheral init → main loop → input decision → output control → UART reporting. Clear symbols, arrows, decisions properly marked.	Basic flowchart present with main blocks (init, loop, input/output). Some labels missing or flow unclear at decision points.	Incomplete flowchart or hand-drawn sketch without proper symbols/logic flow. Major gaps in understanding shown.
PBL Implementation (Code & Hardware)	Code compiles/runs perfectly. All peripherals (GPIO, UART, timer) correctly configured via HAL/CubeMX. Modular structure, proper comments, hardware wiring correct.	Code works with minor issues. Basic peripheral config correct but may have inefficient loops or missing error checks. Hardware functional.	Code has compilation errors or major logic flaws. Peripherals misconfigured. Hardware connections faulty or incomplete.
Demonstration	Full functionality demonstrated — LED behavior perfect, UART messages exactly as specified, confident explanation of working.	Core functionality works (LED responds, UART transmits) but timing/messages have minor issues. Basic explanation provided.	Demo partially works or fails frequently. Cannot explain working properly or show expected outputs.
Practical File	File is complete in all aspects, observation and reflection summary is correct	File is complete in all aspects, observation and reflection summary has some minor issues	File is complete but poorly maintained, reflection summary is not written well

Course Coordinator Name and Signature

CHITKARA UNIVERSITY

Department of Electronics and Communication Engineering

CUIET, Chitkara University Punjab

Course Name: IoT Application Development | **Course Code:** 23EC362

Batch: 2023 | **Credits:** 4

Session: Jan-Jun 2026 | **Semester:** 6th

Date: 23 Feb 2026 | **Class/Group:** IoT Vertical

Lab Evaluation – II

Code: /* USER CODE BEGIN Header */

```c
/**
 ******************************************************************************
 * @file           : main.c
 * @brief          : Main program body
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ******************************************************************************
 */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "stm32l475e_iot01.h"
#include "stm32l475e_iot01_tsensor.h"
#include <stdio.h>
#include <string.h>
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
```

Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development
Batch: 2023
Session: Jan-Jun 2026
Date: 23 Feb 2026

Course Code: 23EC362
Credits: 4
Semester: 6th
Class/Group: IoT Vertical

Lab Evaluation – II

```
#define SERVO_MIN_PULSE 500
#define SERVO_MAX_PULSE 2500
/* USER CODE END PD */
/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables ---------------------------------------------------------*/
I2C_HandleTypeDef hi2c1;
TIM_HandleTypeDef htim2;
UART_HandleTypeDef huart1;
/* USER CODE BEGIN PV */
char uart_buf[100];
float temperature;
uint8_t angle;
char status[10];
/* USER CODE END PV */
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM2_Init(void);
static void MX_USART1_UART_Init(void);
static void MX_I2C1_Init(void);
/* USER CODE BEGIN PFP */
void set_servo_angle(uint8_t angle);
/* USER CODE END PFP */
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
/* USER CODE END 0 */
/**
 * @brief  The application entry point.
 * @retval int
 */
int main(void)
```

Lab Evaluation – II

```
{
/* USER CODE BEGIN 1 */
/* USER CODE END 1 */
/* MCU Configuration--------------------------------------------------------*/
/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();
/* USER CODE BEGIN Init */
/* USER CODE END Init */
/* Configure the system clock */
SystemClock_Config();
/* USER CODE BEGIN SysInit */
/* USER CODE END SysInit */
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM2_Init();
MX_USART1_UART_Init();
MX_I2C1_Init();
/* USER CODE BEGIN 2 */
BSP_TSENSOR_Init();   // initialize onboard temp sensor
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
/* USER CODE END 2 */
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* Read temperature */
  temperature = BSP_TSENSOR_ReadTemp();
  /* Decision logic */
  if (temperature < 33.0)
  {
    angle = 0;
    strcpy(status, "SAFE");
  }
```

Lab Evaluation – II

```
  else
  {
   angle = 90;
   strcpy(status, "WARNING");
  }
  /* Control servo */
  set_servo_angle(angle);
  /* Format telemetry */
  sprintf(uart_buf, "T:%.1fC|ANG:%d|STAT:%s\r\n",
        temperature, angle, status);
  /* Send via UART */
  HAL_UART_Transmit(&huart1,
                (uint8_t*)uart_buf,
                strlen(uart_buf),
                HAL_MAX_DELAY);
  HAL_Delay(5000);
 }
 /* USER CODE END 3 */
}
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
 RCC_OscInitTypeDef RCC_OscInitStruct = {0};
 RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
 /** Configure the main internal regulator output voltage
 */
 if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
 {
  Error_Handler();
 }
```

Lab Evaluation – II

```
/** Initializes the RCC Oscillators according to the specified parameters
* in the RCC_OscInitTypeDef structure.
*/
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
RCC_OscInitStruct.HSIState = RCC_HSI_ON;
RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
RCC_OscInitStruct.PLL.PLLM = 1;
RCC_OscInitStruct.PLL.PLLN = 10;
RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
  Error_Handler();
}
/** Initializes the CPU, AHB and APB buses clocks
*/
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                    |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK)
{
  Error_Handler();
}
}
/**
 * @brief I2C1 Initialization Function
 * @param None
```

Lab Evaluation – II

```
 * @retval None
 */
static void MX_I2C1_Init(void)
{
 /* USER CODE BEGIN I2C1_Init 0 */
 /* USER CODE END I2C1_Init 0 */
 /* USER CODE BEGIN I2C1_Init 1 */
 /* USER CODE END I2C1_Init 1 */
 hi2c1.Instance = I2C1;
 hi2c1.Init.Timing = 0x10D19CE4;
 hi2c1.Init.OwnAddress1 = 0;
 hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
 hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
 hi2c1.Init.OwnAddress2 = 0;
 hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
 hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
 hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
 if (HAL_I2C_Init(&hi2c1) != HAL_OK)
 {
  Error_Handler();
 }
 /** Configure Analogue filter
 */
 if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
 {
  Error_Handler();
 }
 /** Configure Digital filter
 */
 if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK)
 {
  Error_Handler();
 }
```

Lab Evaluation – II

```
 /* USER CODE BEGIN I2C1_Init 2 */
 /* USER CODE END I2C1_Init 2 */
}
/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{
 /* USER CODE BEGIN TIM2_Init 0 */
 /* USER CODE END TIM2_Init 0 */
 TIM_ClockConfigTypeDef sClockSourceConfig = {0};
 TIM_MasterConfigTypeDef sMasterConfig = {0};
 TIM_OC_InitTypeDef sConfigOC = {0};
 /* USER CODE BEGIN TIM2_Init 1 */
 /* USER CODE END TIM2_Init 1 */
 htim2.Instance = TIM2;
 htim2.Init.Prescaler = 79;
 htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
 htim2.Init.Period = 19999;
 htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
 htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
 if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
 {
  Error_Handler();
 }
 sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
 if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
 {
  Error_Handler();
 }
 if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
```

Lab Evaluation – II

```
{
  Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
{
  Error_Handler();
}
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 0;
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
{
  Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */
/* USER CODE END TIM2_Init 2 */
HAL_TIM_MspPostInit(&htim2);
}
/**
 * @brief USART1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART1_UART_Init(void)
{
/* USER CODE BEGIN USART1_Init 0 */
/* USER CODE END USART1_Init 0 */
/* USER CODE BEGIN USART1_Init 1 */
/* USER CODE END USART1_Init 1 */
huart1.Instance = USART1;
```

Lab Evaluation – II

```
huart1.Init.BaudRate = 9600;
huart1.Init.WordLength = UART_WORDLENGTH_8B;
huart1.Init.StopBits = UART_STOPBITS_1;
huart1.Init.Parity = UART_PARITY_NONE;
huart1.Init.Mode = UART_MODE_TX_RX;
huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart1.Init.OverSampling = UART_OVERSAMPLING_16;
huart1.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
huart1.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
if (HAL_UART_Init(&huart1) != HAL_OK)
{
  Error_Handler();
}
/* USER CODE BEGIN USART1_Init 2 */
/* USER CODE END USART1_Init 2 */
}
/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
 GPIO_InitTypeDef GPIO_InitStruct = {0};
 /* USER CODE BEGIN MX_GPIO_Init_1 */
 /* USER CODE END MX_GPIO_Init_1 */
 /* GPIO Ports Clock Enable */
 __HAL_RCC_GPIOC_CLK_ENABLE();
 __HAL_RCC_GPIOH_CLK_ENABLE();
 __HAL_RCC_GPIOB_CLK_ENABLE();
 __HAL_RCC_GPIOA_CLK_ENABLE();
 /*Configure GPIO pin : PB14 */
 GPIO_InitStruct.Pin = GPIO_PIN_14;
```

Lab Evaluation – II

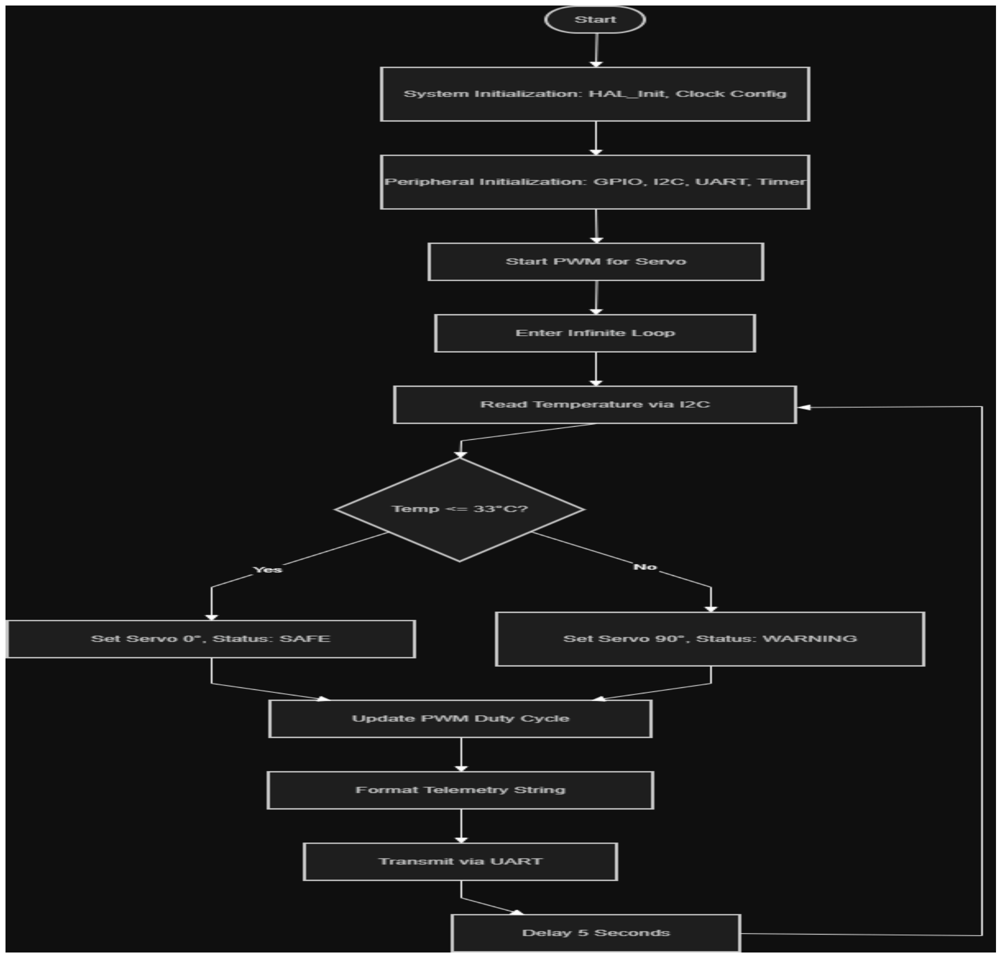
```
 GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
 GPIO_InitStruct.Pull = GPIO_NOPULL;
 HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
 /* USER CODE BEGIN MX_GPIO_Init_2 */
 /* USER CODE END MX_GPIO_Init_2 */
}
/* USER CODE BEGIN 4 */
void set_servo_angle(uint8_t angle)
{
 uint32_t pulse;
 if (angle > 180) angle = 180;
 pulse = SERVO_MIN_PULSE +
        (SERVO_MAX_PULSE - SERVO_MIN_PULSE) * angle / 180;
 __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse);
}
/* USER CODE END 4 */
/**
 * @brief  This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
 /* USER CODE BEGIN Error_Handler_Debug */
 /* User can add his own implementation to report the HAL error return state */
 __disable_irq();
 while (1)
 {
 }
 /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
 * @brief  Reports the name of the source file and the source line number
```

Lab Evaluation – II

```
*        where the assert_param error has occurred.
 * @param  file: pointer to the source file name
 * @param  line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
 /* USER CODE BEGIN 6 */
 /* User can add his own implementation to report the file name and line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
 /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

Lab Evaluation – II

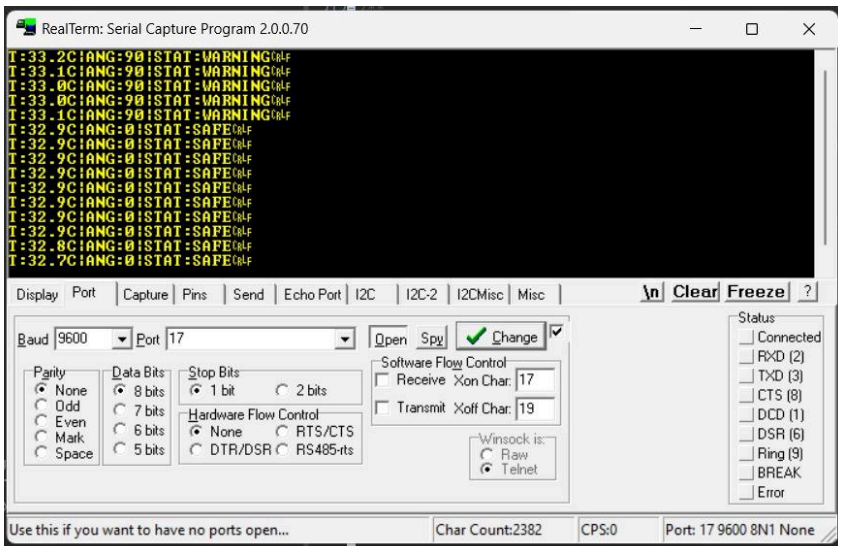
System Flow chart

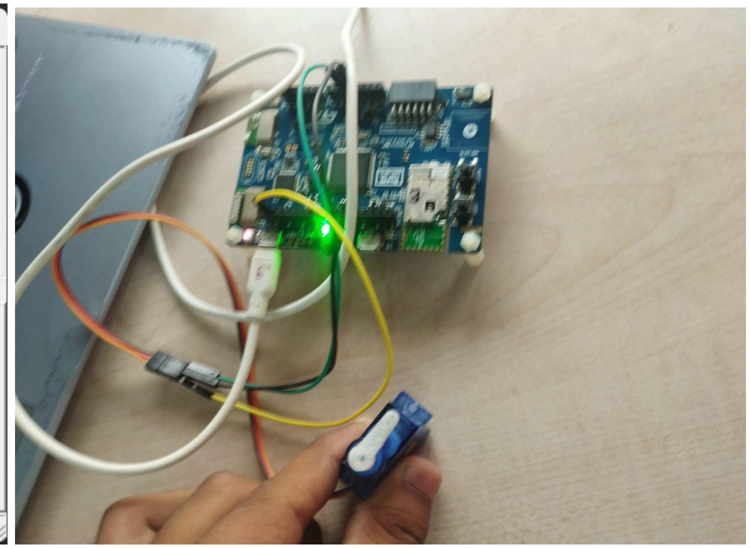


Course Name: IoT Application Development | **Course Code:** 23EC362
Batch: 2023 | **Credits:** 4
Session: Jan-Jun 2026 | **Semester:** 6th
Date: 23 Feb 2026 | **Class/Group:** IoT Vertical

Lab Evaluation – II

Output:





When the Temperature is less than the defined temperature(i.e 33 degree) the servo motor is at 0 degree

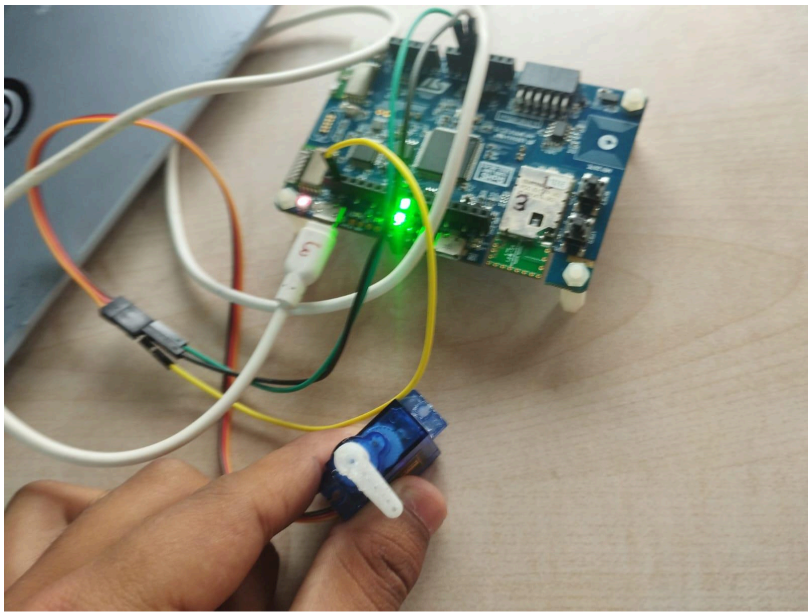
CHITKARA UNIVERSITY

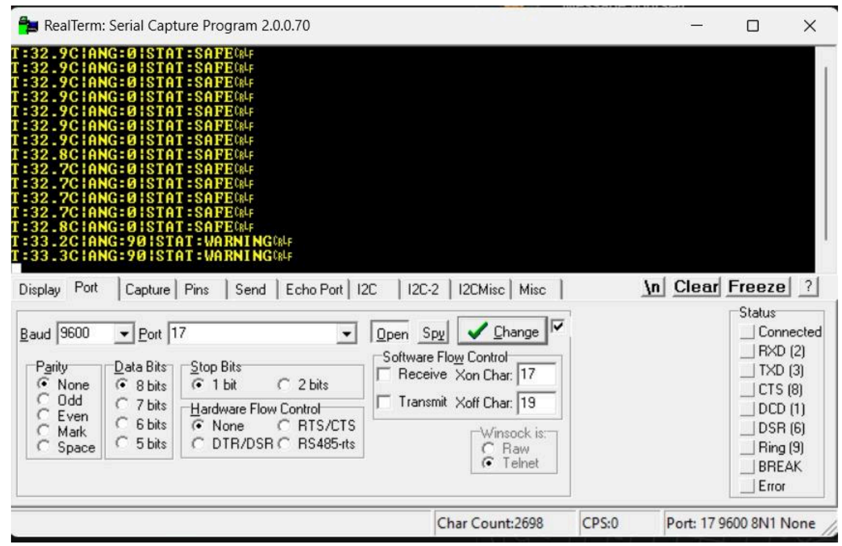
Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development
Batch: 2023
Session: Jan-Jun 2026
Date: 23 Feb 2026

Course Code: 23EC362
Credits: 4
Semester: 6th
Class/Group: IoT Vertical

Lab Evaluation – II





When the temperature is above the defined Temperature the servo motor turn 90 degree.

Lab Evaluation – II

Observation:

During the experiment, the STM32L475 IoT node successfully interfaced with the onboard HTS221 temperature sensor via I2C and continuously monitored real-time temperature values. The system generated a 50 Hz PWM signal using TIM2 to control the angular position of the servo motor.

It was observed that:

- When the temperature was below the threshold value (~33–35°C), the servo motor remained at 0° position, indicating the SAFE state.
- When the temperature exceeded the threshold, the servo motor moved to 90° position, indicating the WARNING state.
- The system correctly formatted and transmitted telemetry data in the form:

  T:xx.xC|ANG:xx|STAT:xxx

  via UART at 9600 bps, which was successfully displayed on RealTerm.

- The output was updated at a controlled interval of 5 seconds, ensuring stable and readable monitoring.

The experimental setup demonstrated proper synchronization between sensor input, decision logic, actuator response, and communication output, as also shown in the output images on pages 15–16 of your file.

Lab Evaluation – II

Learning Outcome:

After performing this experiment, the following outcomes were achieved:

1. **Understanding of Sensor Interfacing**
   Learned how to interface and acquire real-time data from an onboard temperature sensor (HTS221) using the I2C protocol.
2. **PWM Signal Generation and Servo Control**
   Gained practical knowledge of generating PWM signals using timers and controlling the angular position of a servo motor.
3. **Embedded Decision-Making Logic**
   Understood how to implement conditional logic in embedded systems to trigger actions based on sensor data.
4. **UART Communication and Telemetry Design**
   Learned to format and transmit structured data frames over UART for monitoring and diagnostics.
5. **System Integration Skills**
   Developed the ability to integrate multiple peripherals (GPIO, I2C, Timer, UART) into a complete working embedded system.
6. **Debugging and Hardware-Software Interaction**
   Gained experience in troubleshooting issues related to pin mapping, power supply, and peripheral configuration.
7. **Real-Time Embedded System Design**
   Understood how real-time monitoring and control systems are implemented in industrial automation applications.